Tutorial Sheet 2

Memory Management

AUTHOR
Operating Systems Teaching and Research Unit

PUBLISHED
28.05.2024

Introduction

1.1 The MMU

a) What are the tasks of memory management in operating systems?

b) What tasks does the MMU perform? What is the purpose of dividing a virtual address into segment number (or page number) and offset?

c) Is it possible that a virtual address is called for which no physical address exists? If so, how should the MMU handle it?

d) Why is it beneficial to implement the MMU as a hardware component rather than performing the necessary calculations in software?

ⓘ **Solution**

a) There are several tasks involved in memory management:

- Translating virtual addresses into physical ones
- Allocating memory to processes, releasing memory
- Preventing unauthorized memory accesses
- Paging memory to or from secondary storage
- Facilitating shared memory

b)

The MMU converts virtual addresses into physical addresses. It is located between the CPU and memory, as the instructions that are executed on the CPU use virtual addresses, but the corresponding code or data in the main memory can only be accessed using physical addresses. (There are also caches. These can be located on both sides of the MMU. Caches that are located between the processor and MMU must also be able to handle virtual addresses).

To be more precise: the MMU only has to look at one part of the virtual address - this being the part that addresses a segment or a page. The second half of the address, which describes a position within a block, has the same meaning in both address concepts: it is an offset that is valid from the beginning of a memory area and is therefore independent of whether we are in a particular block of the virtual address space or the physical addresses, as both are the same size.

By splitting addresses into two, the size of the mapping tables of the MMU is reduced: there does not have to be an entry in the table for every possible virtual address, but only for every possible address of a segment/page. This is the same reason multi-level page tables exist.

c)

Such a call may occur - it is actually pretty normal. It means that a process wants to access a memory location that is not currently loaded into the main memory. This triggers a page fault: we are trying to access an unallocated memory area. The MMU must therefore first use the first part of the virtual address to determine which segment/page the process is trying to access and load the corresponding segment/page into the main memory. After this operation, the required physical address exists. This situation will therefore occur regularly.

However, it can also happen that the process attempts to access an address for which it has no authorization. This results in an abort due to a segmentation fault.

d)

Speed! Accesses to virtual addresses occur very frequently and must be converted into accesses to physical addresses as quickly as possible, otherwise the entire process execution will be slowed down.

1.2 System Performance

Given a demand paging system that is currently occupied as follows:

- CPU load: 20%
- ratio of page fault for accessing memory: 97,7%
- load of the other I/O devices: 5%

The aim is to optimize the CPU load in order to process all programs fast and efficiently. For each of the following suggestions, evaluate whether it would improve the CPU load (i.e. increase):

1. install a faster CPU
2. install a larger hard disk
3. increase the number of running/ready processes
4. reduce the number of running/ready processes
5. install more main memory
6. install a faster hard disk

ⓘ Solution

First of all, why is the CPU not being efficiently loaded? It is not because of I/O-heavy processes, as 5% is hardly any access to I/O devices. But the access to the background memory (97,7%) is as high as it gets, therefore apparently the CPU cannot compute more because pages have to be constantly reloaded for the processes. This means we have to reduce the page fault rate.

1. Installing a faster CPU does not help, the CPU is not utilized fully and pages are not loaded faster just because the CPU is faster.
2. Installing a larger hard disk doesn't help either. A larger disk does not give us faster access to the disk, therfore virtual memory access would not speed up.
3. Increasing the number of running processes leads to more processes with their own pages sharing the CPU, This would most-likely produce even more page faults.
4. Reducing the number of running processes probably helps. We (hopefully) reduce the competition for memory and thus the page fault rate, which means the CPU can be better utilized.
5. Installing more main memory helps. It reduces the page fault rate and thus provides more efficient CPU access.
6. Installation of a faster hard disk helps, because if we speed up access to the paging disk, the CPU should also receive data faster.

2. Segmentation

For memory management using segmentation, the following segment table is given for a process:

segment	base address	length
`0x0`	`0x528`	320
`0x1`	`0x232`	58
`0x2`	`0x136`	120
`0x3`	`0x2ee`	60

segment	base address	length
`0x4`	`0x33e`	150
`0x5`	`0x3de`	110

Both the base address and the length are given in bytes.

a) How many bytes are available to the process in physical memory?

b) The address is coded on 16 bits. The maximum segment number is 16, how would you split the virtual address between the *segment number* and the *address* within the segment? What is the maximum segment length?

c) The process requests the address `0x31` within segment number 2, what virtual address will be sent to the MMU?

d) At what time is the virtual address from question **c)** determined?

e) If two separate processes each have a memory segment labeled with the same segment number within their address space, how does the operating system ensure that these segments do not overlap in physical memory?

f) The following virtual addresses are now requested by the process. What does the MMU return in each case?

- `0x301e`
- `0x1012`
- `0x207a`
- `0x4095`

ⓘ Solution

a) Simply taking the sum of the last column results in 818 bytes. (320 + 58 + 120 + 60 + 150 + 110 = 818)

b) The segments number go up to `0xf`, we need at least 4 bits to store this value. With 4 bits, we can store segment number from `0x0` `0xf`. We use the remaining 12 bits for the address within the segment.

The address is stored on 12 bits, it ranges between `0x000` and `0xfff`. Therefore, the maximum segment length is `0xfff` + 1 = 4096.

c) We take the segment number `0x2` and use the last 12 bits for the address: `0x2031`

d) Transparently during compilation.

e) In operating systems with segmentation, each process is linked to a unique segment table, which ensures memory isolation by assigning distinct base addresses to each segment. During a context switch, the operating system loads the segment table of the selected process into the Memory Management Unit (MMU). This setup guarantees that even if different processes use the same segment numbers, they will not access the same physical memory locations, since each process operates with its own segment table.

f)

- `0x301e`: the segments is `0x3` and the address is `0x01e`. We take the base address for the segment 3: `0x2ee`, and sum it with the address: the MMU returns `0x30c`
- `0x1012`: segment number is 1, `0x012` + `0x232` = `0x244`
- `0x207a`: segment number is 2, the address is `0x07a` (or 122). But the length of segment 2 is 120, and 122 exceeds the segment length. It leads to an invalid memory access: the MMU sends an segmentation fault interrupt to the CPU.
- `0x4095`: segment number is 4, `0x33e` + `0x095` = `0x3d3`

3. Paging

Important for tutors

During the tutorial, do a reminder of bitwise operation in C. They will need to use them for the next VPL assignment.

- `&` bitwise AND
- `|` bitwise OR
- `~` bitwise NOT (ones' complement)
- `<<` right shift
- `>>` left shift

```c
uint8_t byte = 0b00000011;

// we want to manipulate the 5th bit counting from the least significant bit

// set bit to 1
byte |= (1 << 5);

/* detail of the operation
1 << 5 gives us 0b00100000
byte |= (1 << 5) gives us 0b00100011
*/

// test the bit value
if (byte & (1 << 5)) {
  printf("flag set\n");
}

/* detail of the operation
flag is 0b00100011
1 << 5 gives us 0b00100000
byte & (1 << 5) gives us 0b00100000
the condition is therefore true
*/

// remove the bit (set to to 0)
byte &= ~(1 << 5);

/* detail of the operation
flag is 0b00100011
1 << 5 gives us 0b00100000
~(1 << 5) gives us 0b11011111
byte & ~(1 << 5) gives us 0b00000011
*/
```

Suppose we have a 32 bits system, managing virtual memory with paging. The page size is 4KiB and the TLB is empty.

The system uses a two level page table, the virtual addresses are divided as follows:

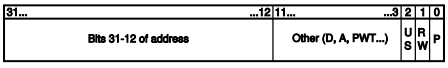
- 10 bits for the index of the first level page table
- 10 bits for the index of the second level page table
- 12 bits for the offset within the page

For instance, the address `0xc01b36e4` is divided like this: `0b1100000000|0110110011|011011100100`

- the index for the first level of the page table is 768
- the index for the second level is 435
- the offset is 1764

The Page Table Entry is composed of 20 bits for the page frame number, and the last 12 bits are used for metadata.

PTE



The last three bits represent the following flags:

- **U/S**: set to 1 if the page can be accessed either in user or supervisor mode
- **R/W**: set to 1 if the page is r/w, 0 if read-only
- **P**: set to 1 if the page is present in memory

Memory pages

1023	0x761c62fc
1022	0xec0fe000
1021	0x044e2fe1
…	…
610	0x3ed733d1
609	0x54826000
608	0xb0b41068
607	0x1432c000
…	…
382	0xc13e4000
381	0x5fc46348
…	…
7	0x94ed9556
6	0x771af000
5	0x554c9d87
4	0x6cb4f481
3	0x0c9a4000
2	0xd9ffb83a
1	0xf0b77ac6
0	0x321e7b42

addr: **0xef2e5000**

1023	0xf0be9fcd
…	…
261	0xf75c68ae
262	0x5433c09f
263	0x53b40b50
264	0xb3556864
265	0xd3e37a21
266	0x5ce775f3
267	0xaee71050
268	0x84ab8b74
269	0x83cb8c1a
270	0x79ae8204
271	0x6b2ba370
…	…
5	0x3c3704fd
4	0x73839cc3
3	0xce6399e0
2	0xd1fd6562
1	0xcd1fe587
0	0xdd02d6e7

addr: **0x771af000**

1023	0xe11d72a3
1022	0x486143b2
1021	0xcaa73b51
1020	0x74b23643
1019	0x8c88cbd3
1018	0x064a0623
1017	0x5a7e0b13
1016	0xb3ff3b53
1015	0xd2da3be1
1014	0x22abdd03
…	…
674	0xa7f0a152
673	0x5e55ba43
672	0x747a7a3f
671	0x61ae7da2
670	0xf29c9732
669	0x05cbf3a1
…	…
1	0x6cd67cb1
0	0x4215b193

addr: **0x1432c000**

1023	0xbdd85d16
…	…
837	0x1384f7af
836	0xcdb54af2
835	0x07959e6c
834	0x45c0c51a
833	0x13db28dd
832	0x0d72ddd7
831	0x6eba6034
…	…
9	0x36556fd1
8	0x1fb73c5c
7	0xbcb8acb7
6	0xb975c46e
5	0x4830d301
4	0x59ff1322
3	0x5c6f1356
2	0x22da0265
1	0x65103266
0	0x3e797c52

addr: **0x54826000**

1023	0x0ed71b28
…	…
979	0xe2931f71
978	0xbf84f8f6
977	0x212c5399
976	0x1fa440ac
975	0xc8b8dda1
974	0x73b5897a
973	0x64c2d6f9
972	0xe6532b3c
971	0xb0a368a1
970	0x910d0807
969	0xdf3f3877
968	0x45dc0a39
967	0x03b373e2
966	0xb869f0bd
…	…
2	0x322991db
1	0xc7051491
0	0x58f86463

addr: **0xc13e4000**

Suppose that a task is running, the page table pointer of that task contains `0xef2e5000`.

Questions

1. The third page displayed above starts at the address `0x1432c000`. What is the associated page number?
2. The value `0x6cb4f481` is stored inside the first page, at the index 4. What is the address at which the value is stored?
3. What is the size of each page table level? Why is it convenient?
4. What is the total size of memory used by the page table for a single address translation?

ⓘ Solution

1. Each page is 4KiB in size, or `0x1000` bytes. We can determine the page number: `0x1432c000` // `0x1000` = `0x1432c`.
2. The value is located in the page number `0xef2e5` at the index 4. Each entry is 4 bytes in size. (because we have a 32 bit system, each the page table needs to store addresses of 32 bits). The address is `0xef2e5000` + (4*4) = `0xef2e5000` + 16 = `0xef2e5000` + `0x10` = `0xef2e5010`. Depending on how the kernel address space is mapped, this could be either a physical or a virtual address.
3. A single page level table is 2^{10} entries of 2^2 bytes (because we are on a 32 bits system). The size of a page table level is 2^2\times 2^{10} = 2^{12} = 4KiB. This is convevient because this is the size of a page.
4. A single translation corresponds to 1 first level page table (4 KiB) and 1 second level page table (4 KiB). Total size is 8 KiB.

Task

The CPU is performing the following actions:

1. In user mode, read at the address `0x97ea0329`
2. In user mode, write to the address `0x5fbc6582`
3. In kernel mode, read at the address `0x5fbcd6f0`
4. In user mode, read at the address `0x5fbd2bad`
5. In user mode, read at the address `0x97ffbc21`
6. In user mode, read at the address `0x97ea0fe2`

For each action, document the steps involved in MMU translation. Explain how the MMU accesses the Page Table Entry associated with the virtual address, translates the virtual address to a physical address, and handles any interrupts or exceptions encountered during the translation process.

TLB

Translation Lookaside Buffer

Page	Page Frame	FLAGS

Finally, update the TLB accordingly: each entry one after the other. To make things easier, don't flush the TLB and suppose that all entries are valid (!= present).

ⓘ Note

On a more realistic system, the TLB contains more flags for caching etc.

The TLB is either flushed between each context switch or contains a **PCID** (Process Context Identifier) to associate each entry with a specific process.

Solution

The page table pointer points to the address `0xef2e5000`.

1. Address `0x97ea0329`

   In binary, the address looks like this:

   `0b 1001011111 1010100000 001100101001`

   The first 10 bits represent the index in the first level page table: 607

   The next 10 bits represent the index in the second level page table: 672

   The last 12 bits represent the offset within the page frame: `0x329` (or 809)

   On the first page table (pointed by the page table pointer), the index 607 gives us the second level page table: `0x1432c000`

   On the second level page table, the index 672 gives us the PTE (Page Table Entry): `0x747a7a3f`

   This PTE in binary is: `0b 01110100011110100111 101000111111`

   The first 20 bits of the PTE represent the page frame number: `0x747a7`

   The last 12 bits represent the flags associated with the page. We can see that the last three bits are set to 1, which mean that the page is:

   1. set to U/S, the page can be accessed in either user or supervisor mode
   2. set to R/W, the page is read/write (it's not important in our case, since the access is a read)
   3. set to P, the page is present in memory

   The page is present: we can store this result in the TLB. We only store the virtual page number (`0x97ea0`), the page frame number (`0x747a7`) and the relevant flags (U/S, R/W and P because they were set to 1).

   The permissions are OK (the page is not protected against user mode).

   Finally, the translation can be determined with the operation: (`page frame` << 12) | `offset`, we obtain `0x747a7329`, this is the value that will be returned by the MMU.

   The translation steps remain the same for the next values.

2. Address `0x5fbc6582`:

   - TLB miss, we perform a full page walk
   - index of the first level page table: 382, gives `0xc13e4000`
   - index of the second level page table: 966, gives PTE `0xb869f0bd`
   - flags are U/S and P
   - page present (P) in memory: we add the translation to the TLB
   - the operation was a write but the page is protected (flag R/W is not set)
   - the MMU triggers a segmentation fault
   - the kernel kills the faulty application

3. Address `0x5fbcd6f0`

    - TLB miss, we perform a full page walk
    - index of the first level page table: 382, gives `0xc13e4000`
    - index of the second level page table: 973, gives PTE `0x64c2d6f9`
    - only P is set: we add the translation to the TLB
    - U/S is not set, only supervisor mode can access the page
    - the read operation was made by the kernel: it runs in supervisor mode, no segmentation fault occurs
    - we can add the translation to the TLB
    - the MMU returns `0x64c2d6f0`

4. Address `0x5fbd2bad`

    - TLB miss, we perform a full page walk
    - index of the first level page table: 382, gives `0xc13e4000`
    - index of the second level page table: 978, gives PTE `0xbf84f8f6`
    - flags are U/S, R/W
    - P is not set, the MMU triggers is a page fault
    - the kernel gets the interrupt and swaps the page back from disk to memory
    - the translation is added to the TLB (with P flag updated)
    - return the address `0xbf84fbad`

5. Address `0x97ffbc21`

    - TLB miss, we perform a full page walk
    - index of the first level page table: 607, gives `0x1432c000`
    - index of the second level page table: 1019, gives PTE `0x8c88cbd3`
    - flags are R/W, P
    - P is set: we add the translation to the TLB
    - U/S is not set, the page is protected, user mode can't access the page
    - MMU triggers a segmentation fault
    - the kernel kill the application that tried to do the memory access

6. Address `0x97ea0fe2`

    - the page frame `0x97ea0` is present in the TLB, this is a hit
    - we supposed all entries to be valid, so we simply check the permissions
    - the flags in the TLB indicate that the page is accessible in user mode (U/S)
    - the MMU returns the translation `0x747a7fe2`

The TLB at the end is filled with:

TLB

Translation Lookaside Buffer

Page	Page Frame	FLAGS
0x97ea0	0x747a7	U/S RW P
0x5fbc6	0xb869f	U/S P
0x5fbcd	0x64c2d	P
0x5fbd2	0xbf84f	U/S R/W P
0x97ffb	0x8c88c	R/W P

Bonus: Segmentation vs. Paging

Given a computer with **16 GiB** main memory and **64-bit** address space. You are considering whether to use segmentation or paging. You have the choice between 3 options.

Note: 1 GiB = 2^{30} bytes, 1 MiB = 2^{20} bytes

Option 1: Paging with pages of 4 bytes in size

1.1 How many entries would there be in the page table?

1.2 Why such a small page size could cause a lot of TLB misses.

1.3 What is the size of the page table for a single translation, with 1 level of page table?

1.4 What about 2 levels of page table?

1.5 What about 7 levels of page table?

1.6 Why is this number of levels problematic?

Solution

1.1 With an address space of 64 bits, there are 2^{64} possible virtual addresses, each containing one byte. These are then divided into pages of 4 byte in size, resulting in \frac{2^{64}}{2^2}=2^{62} pages.

1.2 Since the page size is so small (4), the translation is shared by only 4 virtual addresses. (a single entry for 4 addresses in the page table). As the page size decreases, fewer addresses share the same translation, resulting in a greater variety of translations and consequently, an increased rate of TLB misses.

1.3 Since we have a single level of page table, we need only 1 table of 2^{62} entries. 1 address is 2^3 bytes so we would end up with 2^{65} bytes of memory for a single translation. This is not possible on a 64 bit machine.

1.4 With two level page table, we need to split the 62 bits in two. We end up with two page tables of 2^{31} entries. The total size for a single address translation is at least one 1st-level page table and one 2nd-level page table. We end up with a total size of 2\times 2^{31}\times 2^3 = 2^{35} = 32 GiB

1.5 With seven levels, we need to split the 62 bits in 7: 62=7\times 8 + 6.

We have 6 bits unused (this means that some addresses can't be translated anymore, but this is not a big deal, 2^{56} pages or 2^{58} addresses is already more than enough for each process; on a modern system, processes only have access to 2^{48} virtual memory addresses)

Each of the seven tables contains 2^8 entries of 2^3 bytes, about 2 KiB per table.

7 tables would be 14 KiB in total.

A single translation would require only 14 KiB with 7 level page table.

1.6 We have way too much computational overhead from consulting all the page tables each time.

But at least, there would be virtually no problems with fragmentation: external fragmentation does not occur with paging anyway, and since memory can be requested in multiples of 4 bytes, there is not much internal fragmentation either.

Option 2: Paging with pages of 256 MiB in size

2.1 How many entries would there be in the page table?

2.2 Why is having pages of this size problematic?

Solution

2.1

If we split the entire address space of 2^{64} addresses by group of 256 Mib = 2^{28}, we end up with \frac{2^{64}}{2^{28}}=2^{36} different pages. The page table needs to hold 2^{36} entries.

2.2 Again, there is no problem with external fragmentation. However, there is likely to be a lot of internal fragmentation, as you can only ever request multiples of 256 MiB.

Side note: We should also increase the number of levels in the page table

Option 3: Segmentation with best fit as a strategy for searching for free memory

3.1 Is there any internal / external fragmentation?

> ⓘ **Solution**
>
> **3.1** Internal fragmentation does not occur, as any size can be requested, but external fragmentation does appear. Also, best fit as a strategy is chosen quite arbitrarily here. It would look the same with the other strategies.

Conclusion:

Based on the questions discussed above, what can we conclude about the optimal solution for memory management?

> ⓘ **Solution**
>
> There isn't an optimal solution, and this holds for memory management in general. Improving on one problem usually worsens another. Therefore, in practice, medium-sized page sizes (4 KiB) are chosen as a trade-off (see Linux).